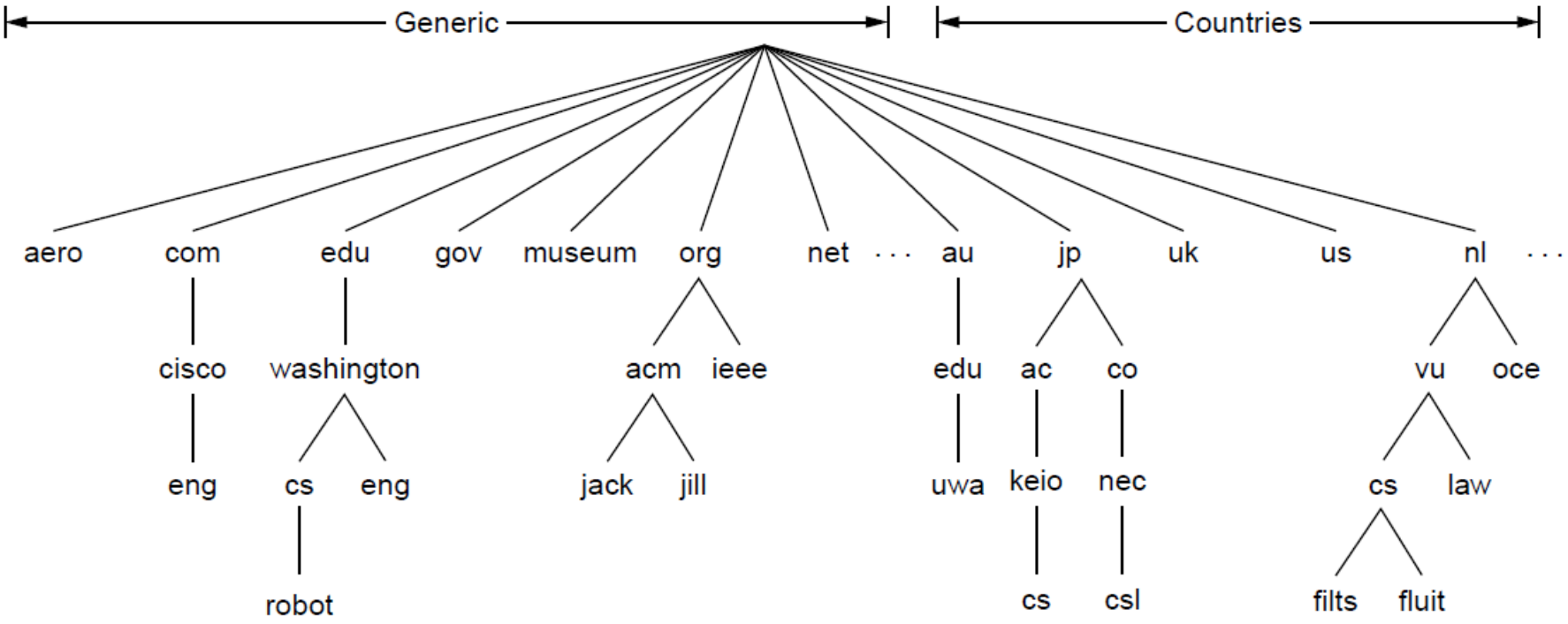
UNIT-V: APPLICATION LAYER

- Domain Name System (DNS)-Name space, Domain resource records, Name servers;
- Electronic mail-Architecture and services, User agent, Message formats, Message transfer, Final delivery;
- The World Wide Web- Architectural overview,
- HTTP.

DNS – The Domain Name System

- The DNS name space
- Domain Resource records
- Name servers

The DNS Name Space (1)



A portion of the Internet domain name space.

The DNS Name Space (2)

Domain	Intended use	Start date	Restricted?
com	Commercial	1985	No
edu	Educational institutions	1985	Yes
gov	Government	1985	Yes
int	International organizations	1988	Yes
mil	Military	1985	Yes
net	Network providers	1985	No
org	Non-profit organizations	1985	No
aero	Air transport	2001	Yes
biz	Businesses	2001	No
coop	Cooperatives	2001	Yes
info	Informational	2002	No
museum	Museums	2002	Yes
name	People	2002	No
pro	Professionals	2002	Yes
cat	Catalan	2005	Yes
jobs	Employment	2005	Yes
mobi	Mobile devices	2005	Yes
tel	Contact details	2005	Yes
travel	Travel industry	2005	Yes
xxx	Sex industry	2010	No

Generic top-level domains

Domain Resource Records (1)

Type	Meaning	Value
SOA	Start of authority	Parameters for this zone
A	IPv4 address of a host	32-Bit integer
AAAA	IPv6 address of a host	128-Bit integer
MX	Mail exchange	Priority, domain willing to accept email
NS	Name server	Name of a server for this domain
CNAME	Canonical name	Domain name
PTR	Pointer	Alias for an IP address
SPF	Sender policy framework	Text encoding of mail sending policy
SRV	Service	Host that provides it
TXT	Text	Descriptive ASCII text

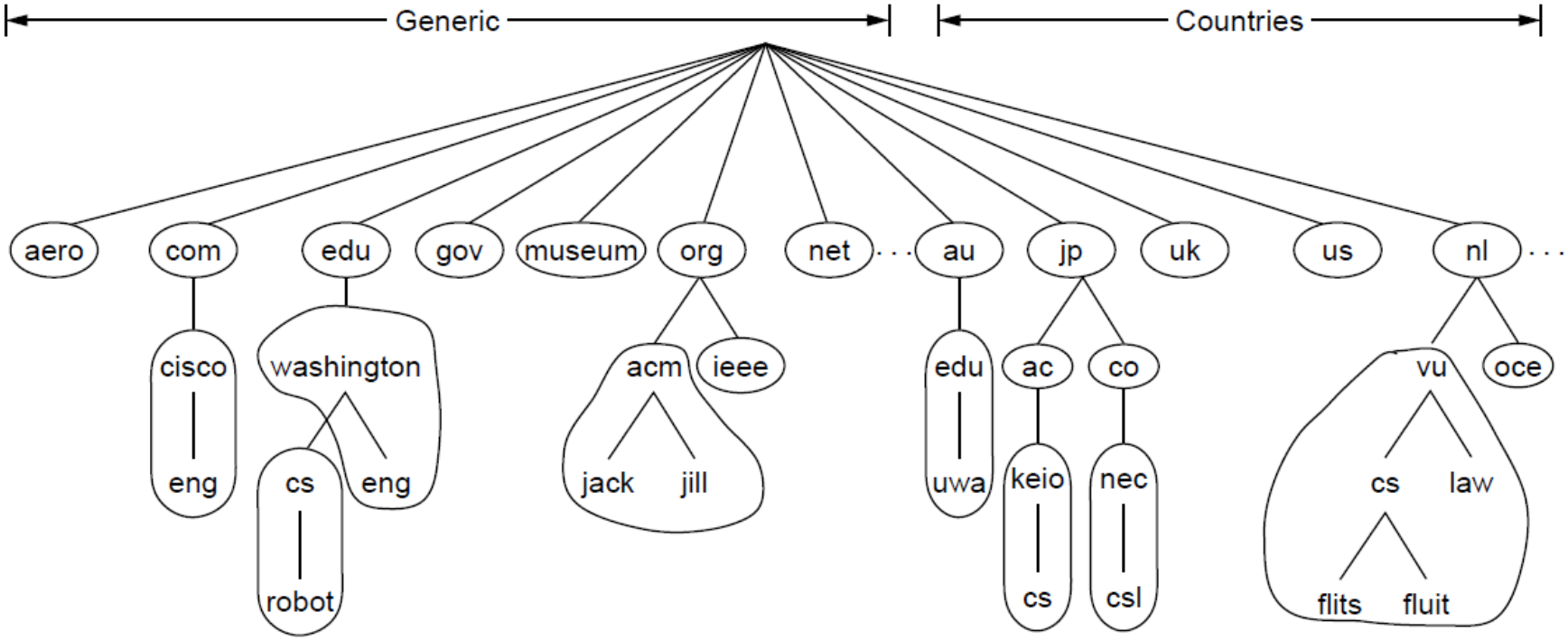
The principal DNS resource record types

Domain Resource Records (2)

```
; Authoritative data for cs.vu.nl
cs.vu.nl.        86400  IN  SOA     star boss (9527,7200,7200,241920,86400)
cs.vu.nl.        86400  IN  MX      1 zephyr
cs.vu.nl.        86400  IN  MX      2 top
cs.vu.nl.        86400  IN  NS      star

star             86400  IN  A       130.37.56.205
zephyr           86400  IN  A       130.37.20.10
top              86400  IN  A       130.37.20.11
www              86400  IN  CNAME   star.cs.vu.nl
ftp              86400  IN  CNAME   zephyr.cs.vu.nl

flits            86400  IN  A       130.37.16.112
flits            86400  IN  A       192.31.231.165
flits            86400  IN  MX      1 flits
flits            86400  IN  MX      2 zephyr
flits            86400  IN  MX      3 top

rowboat                 IN  A       130.37.56.201
                        IN  MX      1 rowboat
                        IN  MX      2 zephyr

little-sister           IN  A       130.37.62.23

laserjet                IN  A       192.31.231.216
```

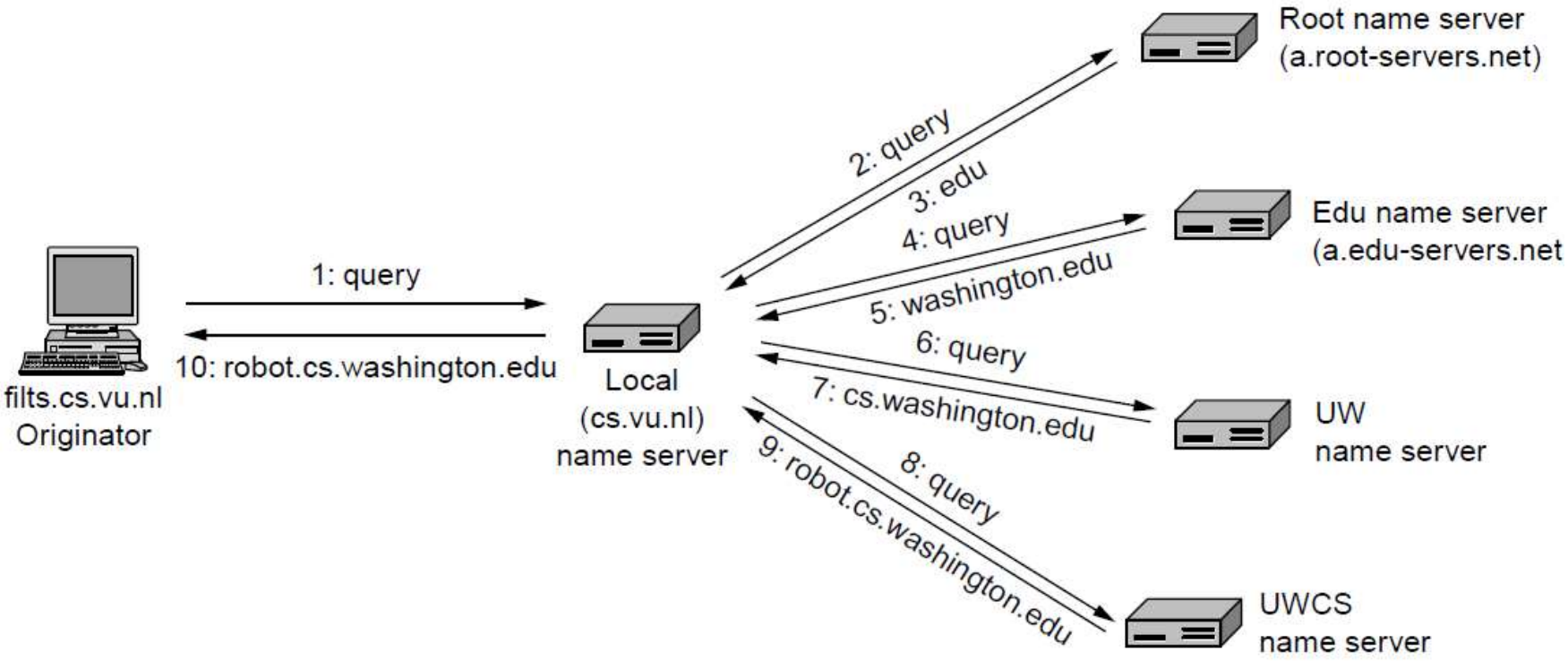
A portion of a possible DNS database for *cs.vu.nl.*

Name Servers (1)



Part of the DNS name space divided into zones (which are circled).

Name Servers (2)

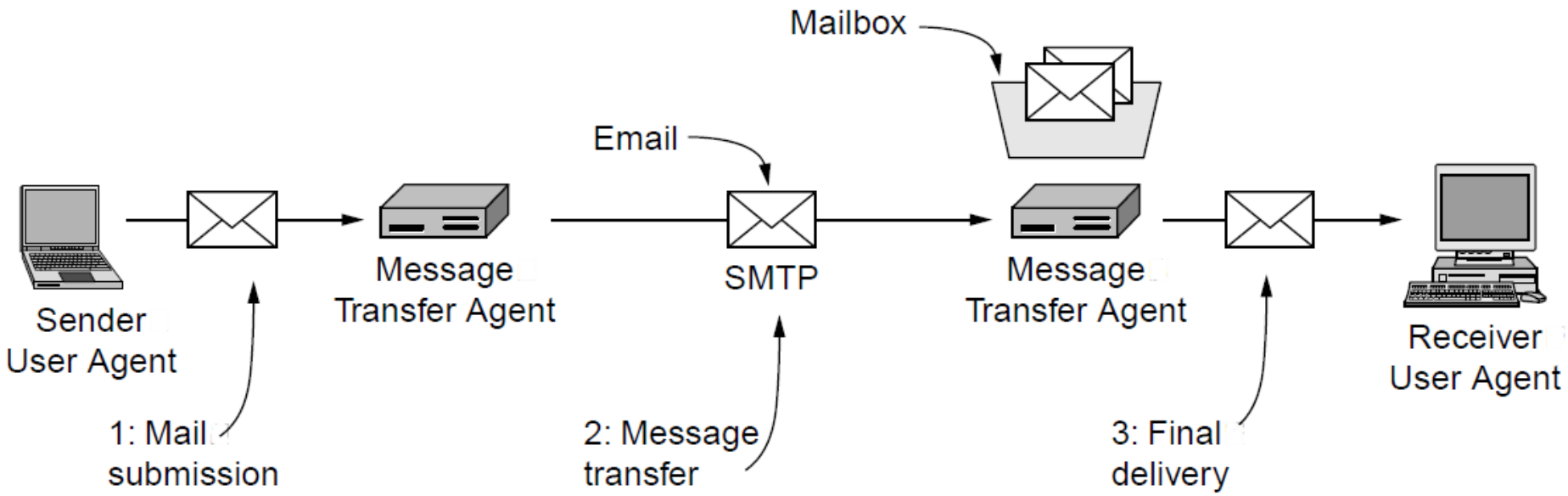


Example of a resolver looking up a remote name in 10 steps.

Electronic Mail

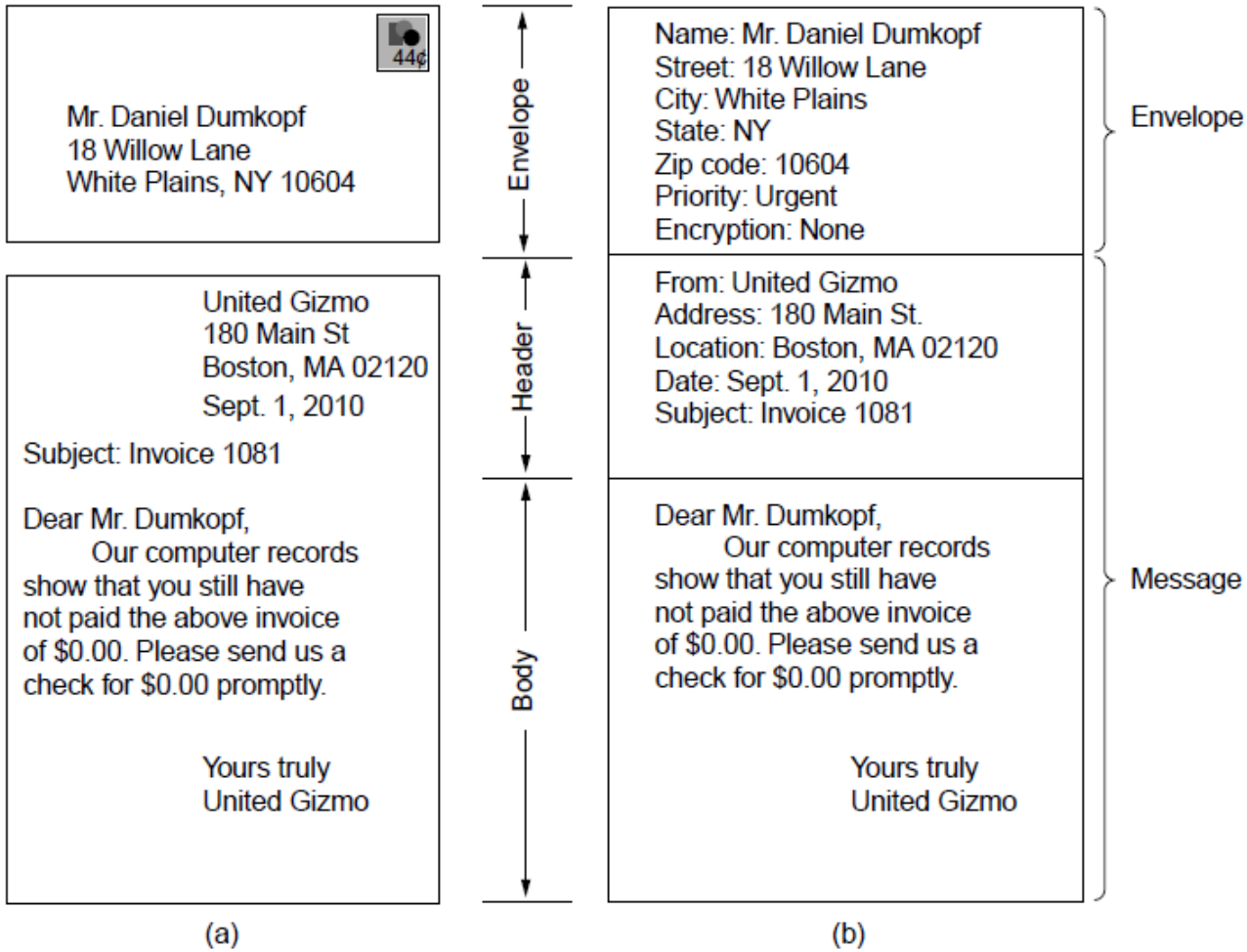
- Architecture and services
- The user agent
- Message formats
- Message transfer
- Final delivery

Architecture and Services (1)



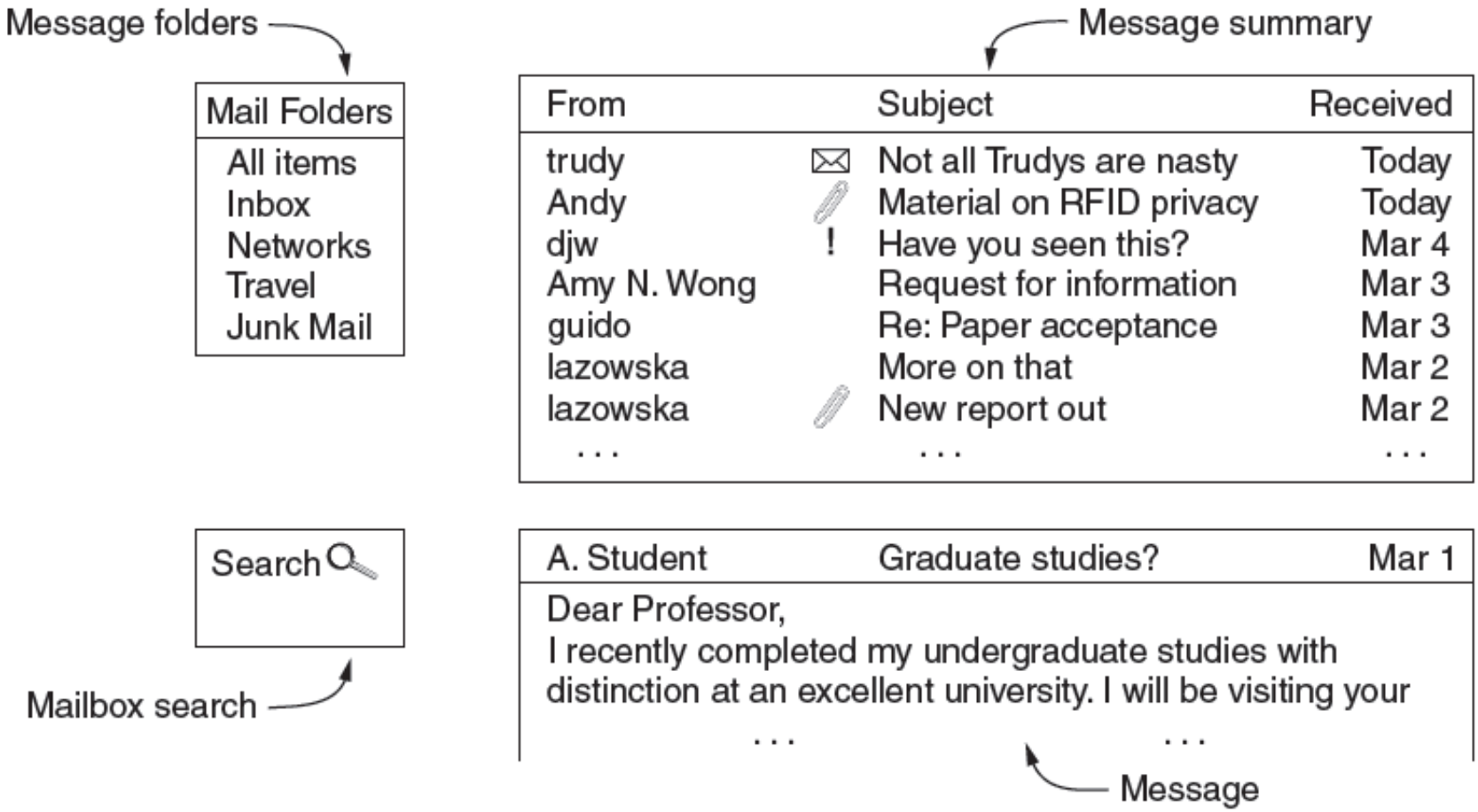
Architecture of the email system

Architecture and Services (2)



Envelopes and messages. (a) Paper mail. (b) Electronic mail.

The User Agent



Typical elements of the user agent interface

Message Formats (1)

Header	Meaning
To:	Email address(es) of primary recipient(s)
Cc:	Email address(es) of secondary recipient(s)
Bcc:	Email address(es) for blind carbon copies
From:	Person or people who created the message
Sender:	Email address of the actual sender
Received:	Line added by each transfer agent along the route
Return-Path:	Can be used to identify a path back to the sender

RFC 5322 header fields related to message transport.

Message Formats (2)

Header	Meaning
Date:	The date and time the message was sent
Reply-To:	Email address to which replies should be sent
Message-Id:	Unique number for referencing this message later
In-Reply-To:	Message-Id of the message to which this is a reply
References:	Other relevant Message-Ids
Keywords:	User-chosen keywords
Subject:	Short summary of the message for the one-line display

Some fields used in the RFC 5322 message header.

Message Formats (3)

Header	Meaning
MIME-Version:	Identifies the MIME version
Content-Description:	Human-readable string telling what is in the message
Content-Id:	Unique identifier
Content-Transfer-Encoding:	How the body is wrapped for transmission
Content-Type:	Type and format of the content

Message headers added by MIME.

Multipurpose Internet Mail Extensions. It is used to extend the capabilities of Internet e-mail protocols such as SMTP.

Message Formats (4)

Type	Example subtypes	Description
text	plain, html, xml, css	Text in various formats
image	gif, jpeg, tiff	Pictures
audio	basic, mpeg, mp4	Sounds
video	mpeg, mp4, quicktime	Movies
model	vrml	3D model
application	octet-stream, pdf, javascript, zip	Data produced by applications
message	http, rfc822	Encapsulated message
multipart	mixed, alternative, parallel, digest	Combination of multiple types

MIME content types and example subtypes.

Message Transfer (1)

```
From: alice@cs.washington.edu
To: bob@ee.uwa.edu.au
MIME-Version: 1.0
Message-Id: <0704760941.AA00747@cs.washington.edu>
Content-Type: multipart/alternative; boundary=qwertyuiopasdfghjklzxcvbnm
Subject: Earth orbits sun integral number of times

This is the preamble. The user agent ignores it. Have a nice day.

--qwertyuiopasdfghjklzxcvbnm
Content-Type: text/html

<p>Happy birthday to you<br>
Happy birthday to you<br>
Happy birthday dear <b> Bob </b><br>
Happy birthday to you</p>
```

A multipart message containing HTML and audio alternatives.

Message Transfer (2)

...

```
--qwertyuiopasdfghjklzxcvbnm
Content-Type: message/external-body;
        access-type="anon-ftp";
        site="bicycle.cs.washington.edu";
        directory="pub";
        name="birthday.snd"

content-type: audio/basic
content-transfer-encoding: base64
--qwertyuiopasdfghjklzxcvbnm--
```

A multipart message containing HTML and audio alternatives.

Message Transfer (3)

```
                    S: 220 ee.uwa.edu.au SMTP service ready
C: HELO abcd.com
                    S: 250 cs.washington.edu says hello to ee.uwa.edu.au
C: MAIL FROM: <alice@cs.washington.edu>
                    S: 250 sender ok
C: RCPT TO: <bob@ee.uwa.edu.au>
                    S: 250 recipient ok
C: DATA
                    S: 354 Send mail; end with "." on a line by itself
C: From: alice@cs.washington.edu
C: To: bob@ee.uwa.edu.au
C: MIME-Version: 1.0
C: Message-Id: <0704760941.AA00747@ee.uwa.edu.au>
C: Content-Type: multipart/alternative; boundary=qwertyuiopasdfghjklzxcvbnm
C: Subject: Earth orbits sun integral number of times
C:
C: This is the preamble. The user agent ignores it. Have a nice day.
C:
C: --qwertyuiopasdfghjklzxcvbnm
C: Content-Type: text/html
C:      . . .
```

Sending a message from *alice@cs.washington.edu* to *bob@ee.uwa.edu.au.*

… Message Transfer (4)

```
C: <p>Happy birthday to you
C: Happy birthday to you
C: Happy birthday dear <bold> Bob </bold>
C: Happy birthday to you
C:
C: --qwertyuiopasdfghjklzxcvbnm
C: Content-Type: message/external-body;
C:         access-type="anon-ftp";
C:         site="bicycle.cs.washington.edu";
C:         directory="pub";
C:         name="birthday.snd"
C:
C: content-type: audio/basic
C: content-transfer-encoding: base64
C: --qwertyuiopasdfghjklzxcvbnm
C: .
                    S: 250 message accepted
C: QUIT
                    S: 221 ee.uwa.edu.au closing connection
```

Sending a message from *alice@cs.washington.edu* to *bob@ee.uwa.edu.au.*

Message Transfer (5)

Keyword	Description
AUTH	Client authentication
BINARYMIME	Server accepts binary messages
CHUNKING	Server accepts large messages in chunks
SIZE	Check message size before trying to send
STARTTLS	Switch to secure transport (TLS; see Chap. 8)
UTF8SMTP	Internationalized addresses

Some SMTP extensions.

- The **Internet Message Access Protocol** Version 4rev2 (IMAP4rev2) allows a client to access and manipulate electronic mail messages on a server.
- IMAP4rev2 permits manipulation of **mailboxes** (remote message folders) in a way that is functionally equivalent to local folders.
- IMAP4rev2 also provides the capability for an offline client to resynchronize with the server.

Final Delivery (1)

Command	Description
CAPABILITY	List server capabilities
STARTTLS	Start secure transport (TLS; see Chap. 8)
LOGIN	Log on to server
AUTHENTICATE	Log on with other method
SELECT	Select a folder
EXAMINE	Select a read-only folder
CREATE	Create a folder
DELETE	Delete a folder
RENAME	Rename a folder
SUBSCRIBE	Add folder to active set
UNSUBSCRIBE	Remove folder from active set

...

IMAP (version 4) commands.

…

Final Delivery (2)

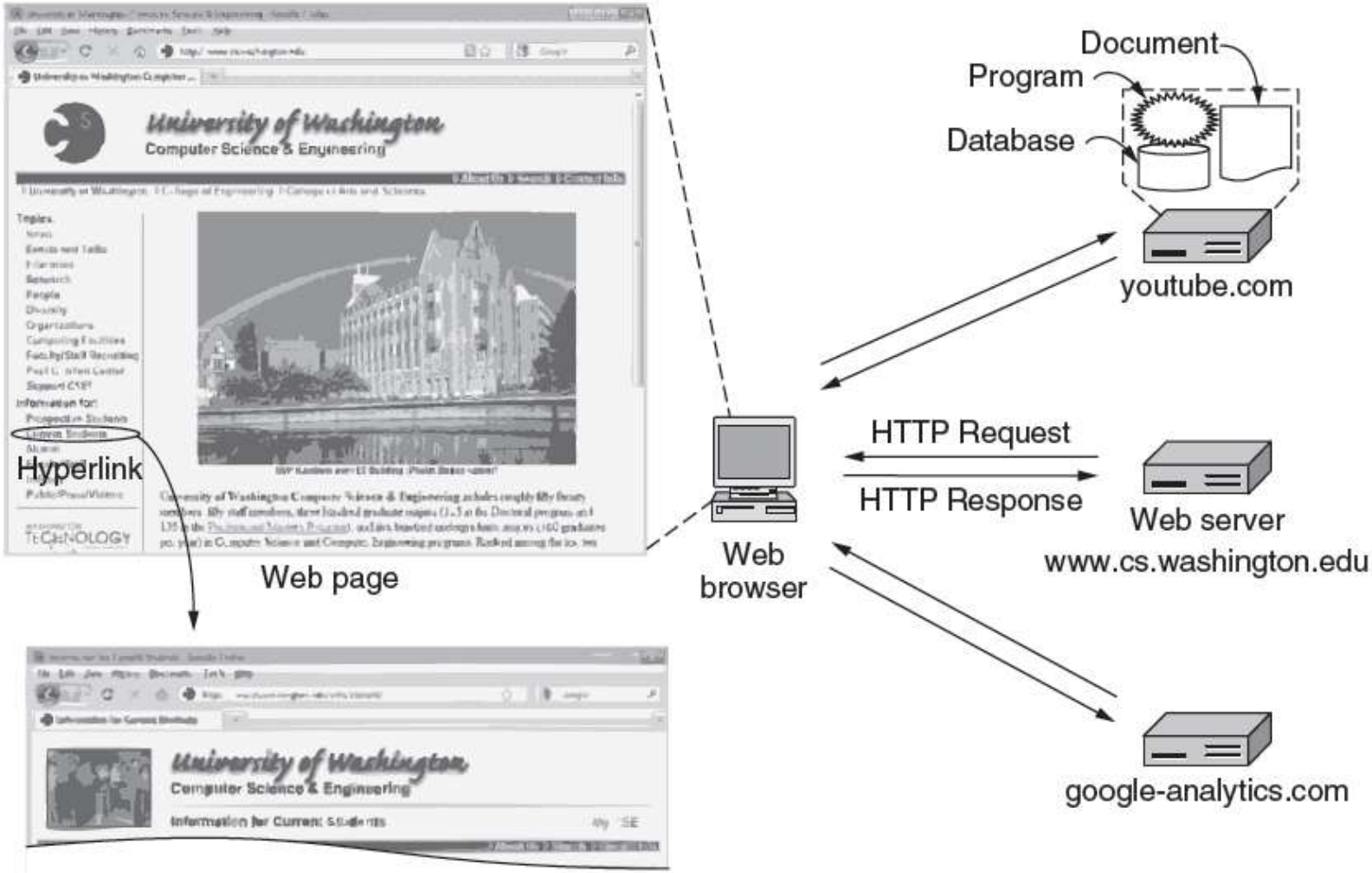
LIST	List the available folders
LSUB	List the active folders
STATUS	Get the status of a folder
APPEND	Add a message to a folder
CHECK	Get a checkpoint of a folder
FETCH	Get messages from a folder
SEARCH	Find messages in a folder
STORE	Alter message flags
COPY	Make a copy of a message in a folder
EXPUNGE	Remove messages flagged for deletion
UID	Issue commands using unique identifiers
NOOP	Do nothing
CLOSE	Remove flagged messages and close folder
LOGOUT	Log out and close connection

IMAP (version 4) commands.

The World Wide Web

- Architectural overview
- Static web pages
- Dynamic web pages, web applications
- The hypertext transfer protocol
- The mobile web
- Web search

Architectural Overview (1)



Architecture of the Web.

Architectural Overview (2)

Three questions had to be answered before a selected page could be displayed:

1. What is the page called?
2. Where is the page located?
3. How can the page be accessed?

Architectural Overview (3)

Steps that occur when link is selected:

1. Browser determines the URL
2. Browser asks DNS for the IP address of the server
3. DNS replies
4. The browser makes a TCP connection
5. Sends HTTP request for the page
6. Server sends the page as HTTP response
7. Browser fetches other URLs as needed
8. The browser displays the page
9. The TCP connections are released

Architectural Overview (4)

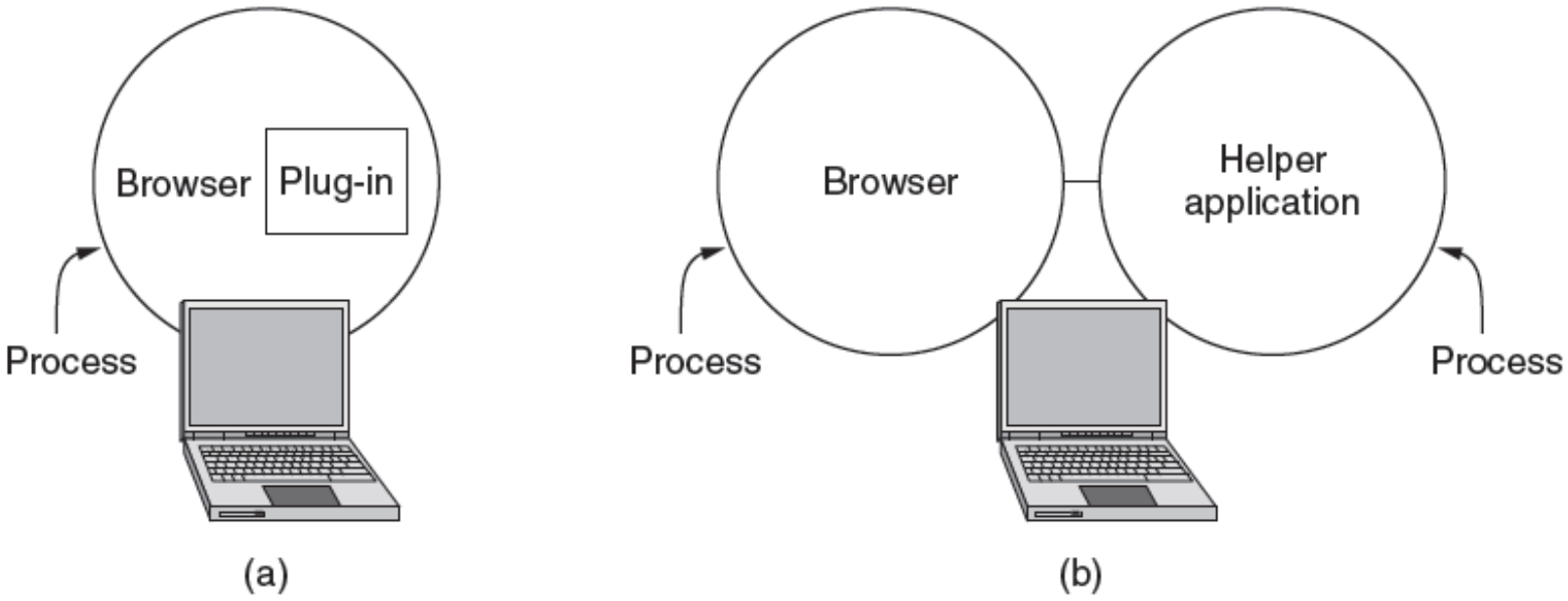
Name	Used for	Example
http	Hypertext (HTML)	http://www.ee.uwa.edu/~rob/
https	Hypertext with security	https://www.bank.com/accounts/
ftp	FTP	ftp://ftp.cs.vu.nl/pub/minix/README
file	Local file	file:///usr/suzanne/prog.c
mailto	Sending email	mailto:JohnUser@acm.org
rtsp	Streaming media	rtsp://youtube.com/montypython.mpg
sip	Multimedia calls	sip:eve@adversary.com
about	Browser information	about:plugins

Some common URL schemes.

The **Session Initiation Protocol (SIP)** is a signaling protocol used for initiating, maintaining, and terminating communication sessions that include voice, video and messaging applications. SIP is used in Internet telephony, in private IP telephone systems, as well as mobile phone calling over LTE (VoLTE-Voice over Long-Term Evolution).

Real Time Streaming Protocol (RTSP) is an application-level network communication system that transfers real-time data from multimedia to an endpoint device by communicating directly with the server streaming the data.

Architectural Overview (5)



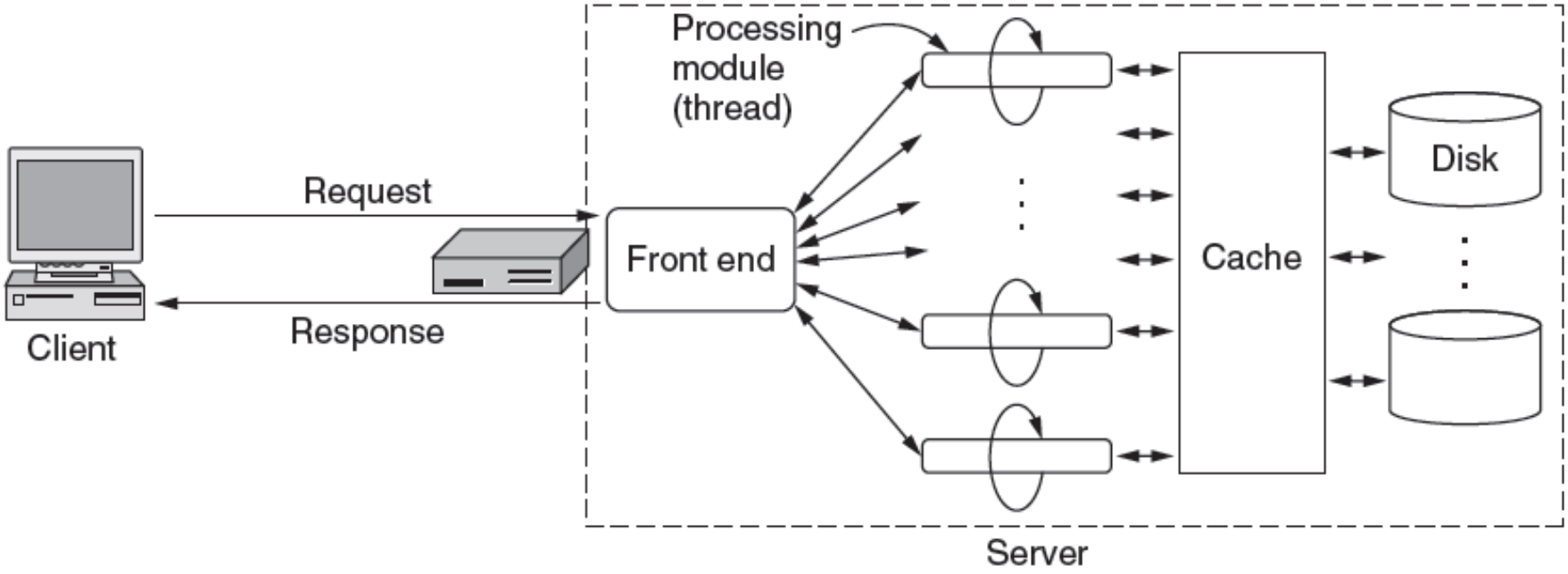
(a) A browser plug-in. (b) A helper application.

Architectural Overview (6)

Steps server performs in main loop

1. Accept a TCP connection from client
2. Get path to page, name of file requested.
3. Get the file (from disk).
4. Send contents of the file to the client.
5. Release the TCP connection.

Architectural Overview



A multithreaded Web server with a front end and processing modules.

Architectural Overview

A processing module performs a series of steps:

1. Resolve name of Web page requested.
2. Perform access control on the Web page.
3. Check the cache.
4. Fetch requested page from disk or run program
5. Determine the rest of the response
6. Return the response to the client.
7. Make an entry in the server log.

Architectural Overview

Domain	Path	Content	Expires	Secure
toms-casino.com	/	CustomerID=297793521	15-10-10 17:00	Yes
jills-store.com	/	Cart=1-00501;1-07031;2-13721	11-1-11 14:22	No
aportal.com	/	Prefs=Stk:CSCO+ORCL;Spt:Jets	31-12-20 23:59	No
sneaky.com	/	UserID=4627239101	31-12-19 23:59	No

Some examples of cookies

Static Web Pages (1)

```
<html>
<head> <title> AMALGAMATED WIDGET, INC. </title> </head>
<body> <h1> Welcome to AWI's Home Page </h1>
<img src="http://www.widget.com/images/logo.gif" ALT="AWI Logo"> <br>
We are so happy that you have chosen to visit <b> Amalgamated Widget's</b>
home page. We hope <i> you </i> will find all the information you need here.
<p>Below we have links to information about our many fine products.
You can order electronically (by WWW), by telephone, or by email. </p>
<hr>
<h2> Product information </h2>
<ul>
   <li> <a href="http://widget.com/products/big"> Big widgets </a> </li>
   <li> <a href="http://widget.com/products/little"> Little widgets </a> </li>
</ul>
<h2> Contact information </h2>
<ul>
   <li> By telephone: 1-800-WIDGETS </li>
   <li> By email: info@amalgamated-widget.com </li>
</ul>
</body>
</html>
```

The HTML for a sample Web page.

Static Web Pages (2)

Welcome to AWI's Home Page

We are so happy that you have chosen to visit **Amalgamated Widget's** home page. We hope *you* will find all the information you need here.

Below we have links to information about our many fine products. You can order electronically (by WWW), by telephone, or by email.

Product Information

- Big widgets
- Little widgets

Contact information

- By telephone: 1-800-WIDGETS
- By email: info@amalgamated-widget.com

The formatted page.

Static Web Pages (3)

Item	HTML 1.0	HTML 2.0	HTML 3.0	HTML 4.0	HTML 5.0
Hyperlinks	x	x	x	x	x
Images	x	x	x	x	x
Lists	x	x	x	x	x
Active maps & images		x	x	x	x
Forms		x	x	x	x
Equations			x	x	x
Toolbars			x	x	x
Tables			x	x	x
Accessibility features				x	x
Object embedding				x	x
Style sheets				x	x
Scripting				x	x
Video and audio					x
Inline vector graphics					x
XML representation					x
Background threads					x
Browser storage					x
Drawing canvas					x

Some differences between HTML versions.

Static Web Pages (4)

```
<html>
<head> <title> AWI CUSTOMER ORDERING FORM </title> </head>
<body>
<h1> Widget Order Form </h1>
<form ACTION="http://widget.com/cgi-bin/order.cgi" method=POST>
<p> Name <input name="customer" size=46> </p>
<p> Street address <input name="address" size=40> </p>
<p> City <input name="city" size=20> State <input name="state" size =4>
Country <input name="country" size=10> </p>
<p> Credit card # <input name="cardno" size=10>
Expires <input name="expires" size=4>
M/C <input name="cc" type=radio value="mastercard">
VISA <input name="cc" type=radio value="visacard"> </p>
<p> Widget size Big <input name="product" type=radio value="expensive">
Little <input name="product" type=radio value="cheap">
Ship by express courier <input name="express" type=checkbox> </p>
<p><input type=submit value="Submit order"> </p>
Thank you for ordering an AWI widget, the best widget money can buy!
</form>
</body>
</html>
```

Static Web Pages (5)

Widget Order Form

Name

Street address

City State Country

Credit card # Expires M/C Visa

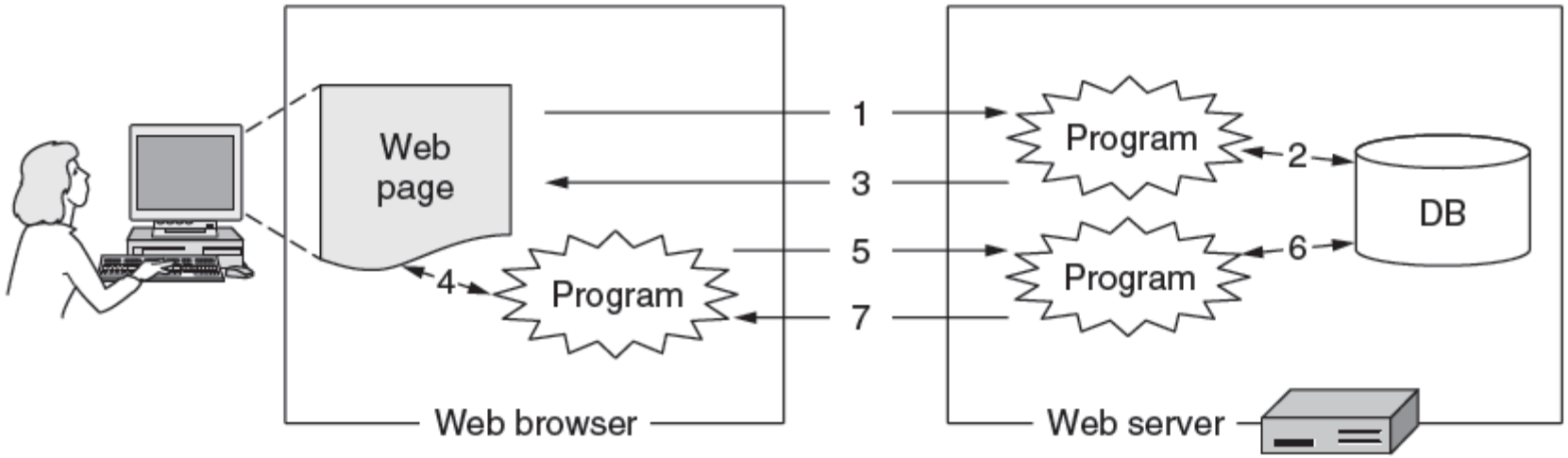
Widget size Big Little Ship by express courier

Submit order

Thank you for ordering an AWI widget, the best widget money can buy!

The formatted page.

Dynamic Web Pages, Web Applications (1)



Dynamic pages

Dynamic Web Pages, Web Applications (2)

```
<html>
<body>
<form action="action.php" method="post">
<p> Please enter your name: <input type="text" name="name"> </p>
<p> Please enter your age: <input type="text" name="age"> </p>
<input type="submit">
</form>
</body>
</html>
```

(a)

```
<html>
<body>
<h1> Reply: </h1>
Hello <?php echo $name; ?>.
Prediction: next year you will be <?php echo $age + 1; ?>
</body>
</html>
```

(b)

```
<html>
<body>
<h1> Reply: </h1>
Hello Barbara.
Prediction: next year you will be 33
</body>
</html>
```

(c)

(a) A Web page containing a form.

(b) A PHP script for handling the output of the form.

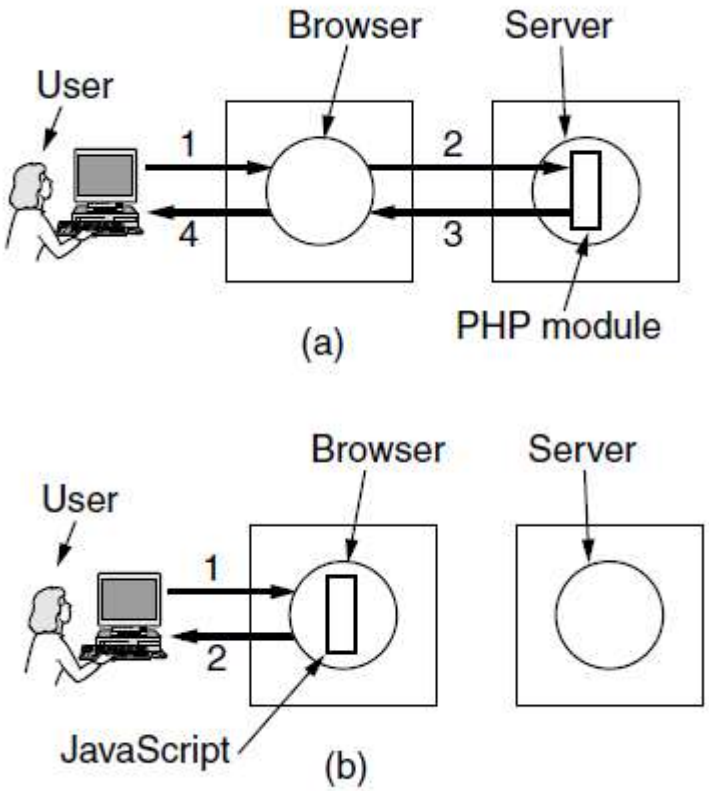
(c) Output from the PHP script when the inputs are “Barbara” and “32”, respectively.

Dynamic Web Pages, Web Applications (3)

```
<html>
<head>
<script language="javascript" type="text/javascript">
function response(test_form) {
   var person = test_form.name.value;
   var years = eval(test_form.age.value) + 1;
   document.open();
   document.writeln("<html> <body>");
   document.writeln("Hello " + person + ".<br>");
   document.writeln("Prediction: next year you will be " + years + ".");
   document.writeln("</body> </html>");
   document.close();
}
</script>
</head>

<body>
<form>
Please enter your name: <input type="text" name="name">
<p>
Please enter your age: <input type="text" name="age">
<p>
<input type="button" value="submit" onclick="response(this.form)">
</form>
</body>
</html>
```

Use of JavaScript for processing a form.

Dynamic Web Pages, Web Applications (4)



(a)Server-side scripting with PHP.

(b)Client-side scripting withJavaScript.

Dynamic Web Pages, Web Applications (5)

AJAX Technologies

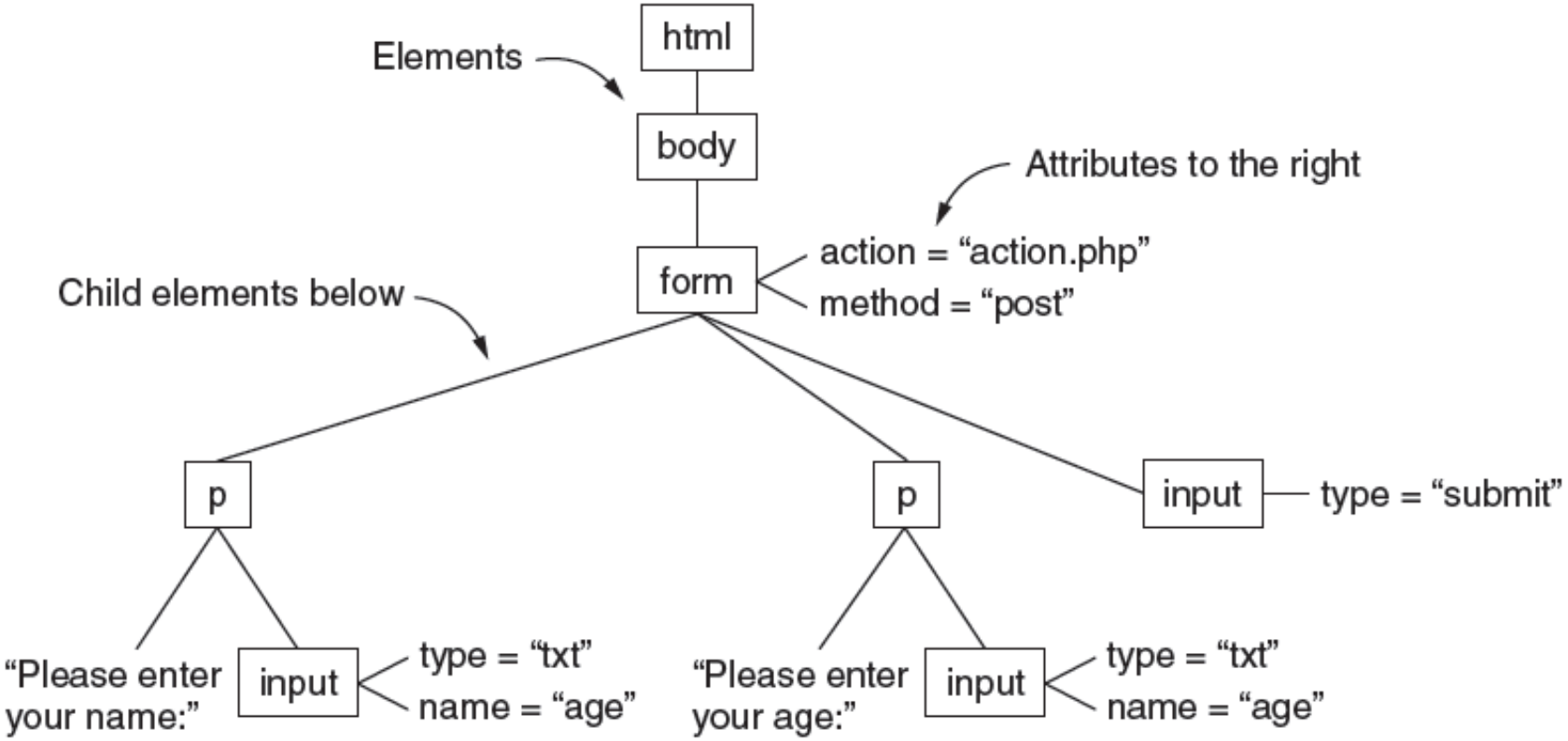
Asynchronous JavaScript And XML.

- **AJAX** is not a programming language.
- **AJAX** just uses a combination of:

A **browser** built-in XMLHttpRequest object (to request data from a web server) J**avaScript and HTML DOM** (to display or use the data)

1. **HTML and CSS:** present information as pages.
2. **DOM:** change parts of pages while they are viewed.
3. **XML:** let programs exchange data with the server.
4. An asynchronous way to send and retrieve XML data.
5. JavaScript as a language to bind all this together.

Dynamic Web Pages, Web Applications (6)



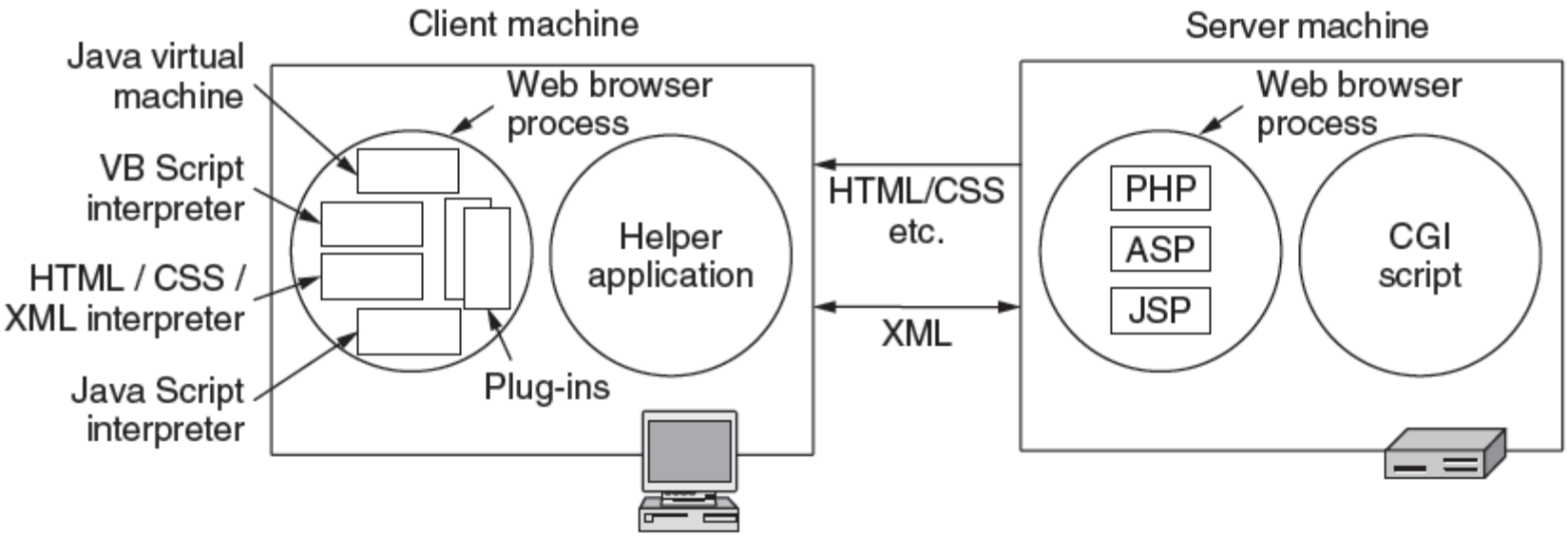
The DOM tree for the HTML in Fig. 7-30(a).

Dynamic Web Pages, Web Applications

```
<?xml version="1.0" ?>

<book_list>

<book>
   <title> Human Behavior and the Principle of Least Effort </title>
   <author> George Zipf </author>
   <year> 1949 </year>
</book>

<book>
   <title> The Mathematical Theory of Communication </title>
   <author> Claude E. Shannon </author>
   <author> Warren Weaver </author>
   <year> 1949 </year>
</book>

<book>
   <title> Nineteen Eighty-Four </title>
   <author> George Orwell </author>
   <year> 1949 </year>
</book>

</book_list>
```

A simple XML document.

Dynamic Web Pages, Web Applications (8)



Various technologies used to generate dynamic pages.

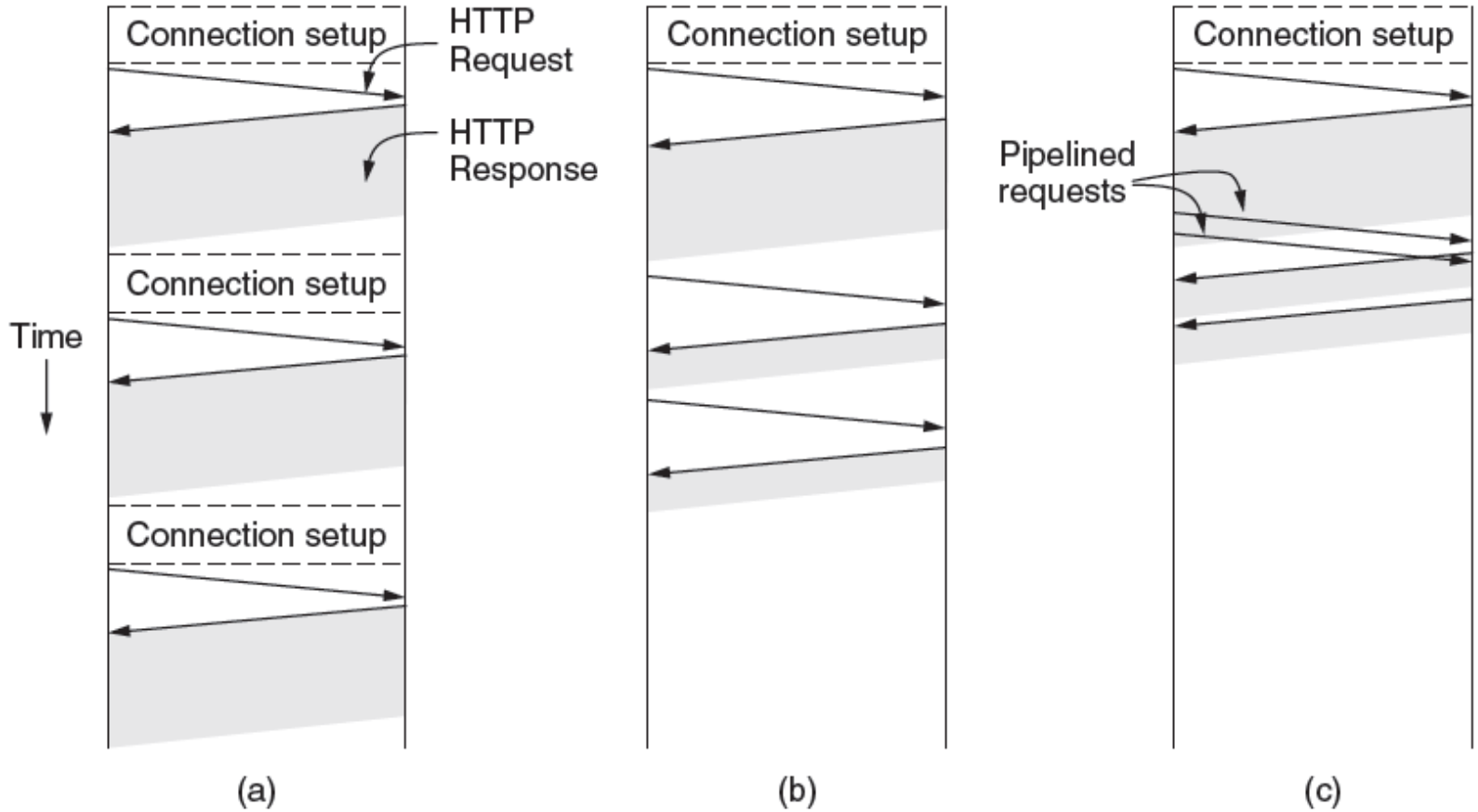
Client Machine Scripting Languages

- **VBScript ("Microsoft Visual Basic Scripting Edition")** is **an Active Scripting language developed by Microsoft that is modeled on Visual Basic**.
- It allows Microsoft Windows system administrators to generate powerful tools for managing computers with **error handling, subroutines, and other advanced programming constructs.**
- **JS-Interpreter** is a sandboxed **JavaScript interpreter** written in JavaScript. It allows for execution of arbitrary **JavaScript code line** by line. Execution is completely isolated from the main JavaScript environment.
- **XML (Extensible Markup Language)** is **a markup language similar to HTML, but without predefined tags to use**. Instead, you define your own tags designed specifically for your needs. This is a powerful way to **store data in a format that can be stored, searched, and shared**.
- **XML was designed to carry data** - with focus on what data is
- **HTML was designed to display data** - with focus on how data looks
- XML tags are not predefined like HTML tags are

Server Machine Scripting Languages

- **CGI** stands for **Common Gateway Interface** and **provides an interface between the HTTP server and programs generating web content**. These programs are better known as CGI scripts. They are written in a scripting language.
- **PHP: Hypertext Preprocessor:** PHP is a widely-used, open source scripting language. PHP scripts are executed on the server. PHP is free to download and use.
- **ASP is Active Server Pages:** ASP is a server side Web building scripting engine. It is generally a server page which includes embedded programming. On the Windows server, the programs are being processed.
- **JSP** is **Java Server Pages** is a collection of technologies that helps software developers create dynamically generated web pages based on HTML, XML, SOAP, or other document types.
- **SOAP (Simple Object Access Protocol)** is a message protocol that enables the distributed elements of an application to communicate. SOAP can be carried over a variety of standard protocols, including the web-related Hypertext Transfer Protocol (HTTP).

The HyperText Transfer Protocol (1)



HTTP with (a) multiple connections and sequential requests.
(b) A persistent connection and sequential requests.
(c) A persistent connection and pipelined requests.

The HyperText Transfer Protocol (2)

Method	Description
GET	Read a Web page
HEAD	Read a Web page's header
POST	Append to a Web page
PUT	Store a Web page
DELETE	Remove the Web page
TRACE	Echo the incoming request
CONNECT	Connect through a proxy
OPTIONS	Query options for a page

The built-in HTTP request methods.

The HyperText Transfer Protocol (3)

Code	Meaning	Examples
1xx	Information	100 = server agrees to handle client's request
2xx	Success	200 = request succeeded; 204 = no content present
3xx	Redirection	301 = page moved; 304 = cached page still valid
4xx	Client error	403 = forbidden page; 404 = page not found
5xx	Server error	500 = internal server error; 503 = try again later

The status code response groups

The HyperText Transfer Protocol (4)

Header	Type	Contents
User-Agent	Request	Information about the browser and its platform
Accept	Request	The type of pages the client can handle
Accept-Charset	Request	The character sets that are acceptable to the client
Accept-Encoding	Request	The page encodings the client can handle
Accept-Language	Request	The natural languages the client can handle
If-Modified-Since	Request	Time and date to check freshness
If-None-Match	Request	Previously sent tags to check freshness
Host	Request	The server's DNS name
Authorization	Request	A list of the client's credentials
Referer	Request	The previous URL from which the request came
Cookie	Request	Previously set cookie sent back to the server
Set-Cookie	Response	Cookie for the client to store
Server	Response	Information about the server

...

Some HTTP message headers.

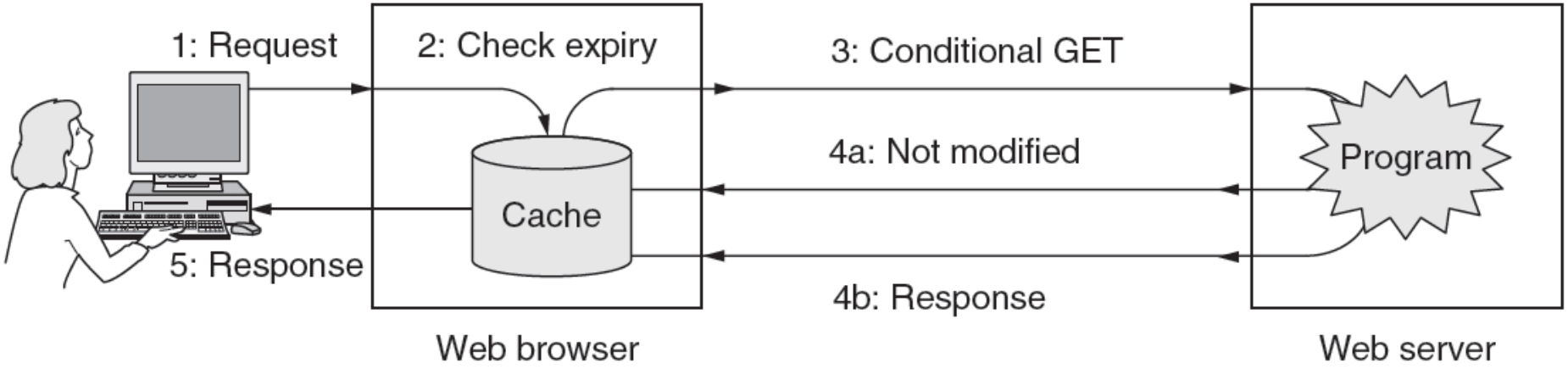
The HyperText Transfer Protocol (5)

…

Content-Encoding	Response	How the content is encoded (e.g., *gzip*)
Content-Language	Response	The natural language used in the page
Content-Length	Response	The page's length in bytes
Content-Type	Response	The page's MIME type
Content-Range	Response	Identifies a portion of the page's content
Last-Modified	Response	Time and date the page was last changed
Expires	Response	Time and date when the page stops being valid
Location	Response	Tells the client where to send its request
Accept-Ranges	Response	Indicates the server will accept byte range requests
Date	Both	Date and time the message was sent
Range	Both	Identifies a portion of a page
Cache-Control	Both	Directives for how to treat caches
ETag	Both	Tag for the contents of the page
Upgrade	Both	The protocol the sender wants to switch to

Some HTTP message headers.

The HyperText Transfer Protocol (6)



HTTP caching.

Network Security

Network Security

Adversary	Goal
Student	To have fun snooping on people's email
Cracker	To test out someone's security system; steal data
Sales rep	To claim to represent all of Europe, not just Andorra
Businessman	To discover a competitor's strategic marketing plan
Ex-employee	To get revenge for being fired
Accountant	To embezzle money from a company
Stockbroker	To deny a promise made to a customer by email
Con man	To steal credit card numbers for sale
Spy	To learn an enemy's military or industrial secrets
Terrorist	To steal germ warfare secrets

Some people who cause security problems and why.

- All these issues **(secrecy, authentication, nonrepudiation, and integrity control)** occur in traditional systems, too, but with some significant differences.
- **Integrity and secrecy** are achieved by using registered mail and locking documents up.
- In the **network layer**, firewalls can be installed to keep good packets and bad packets out. IP security also functions in this layer.
- In the **transport layer**, entire connections can be encrypted, end to end, that is, process to process. For maximum security, end-to-end security is required.
- Finally, issues such as **user authentication and nonrepudiation** can only be handled in the application layer.

Cryptography

- Introduction
- Substitution ciphers
- Transposition ciphers

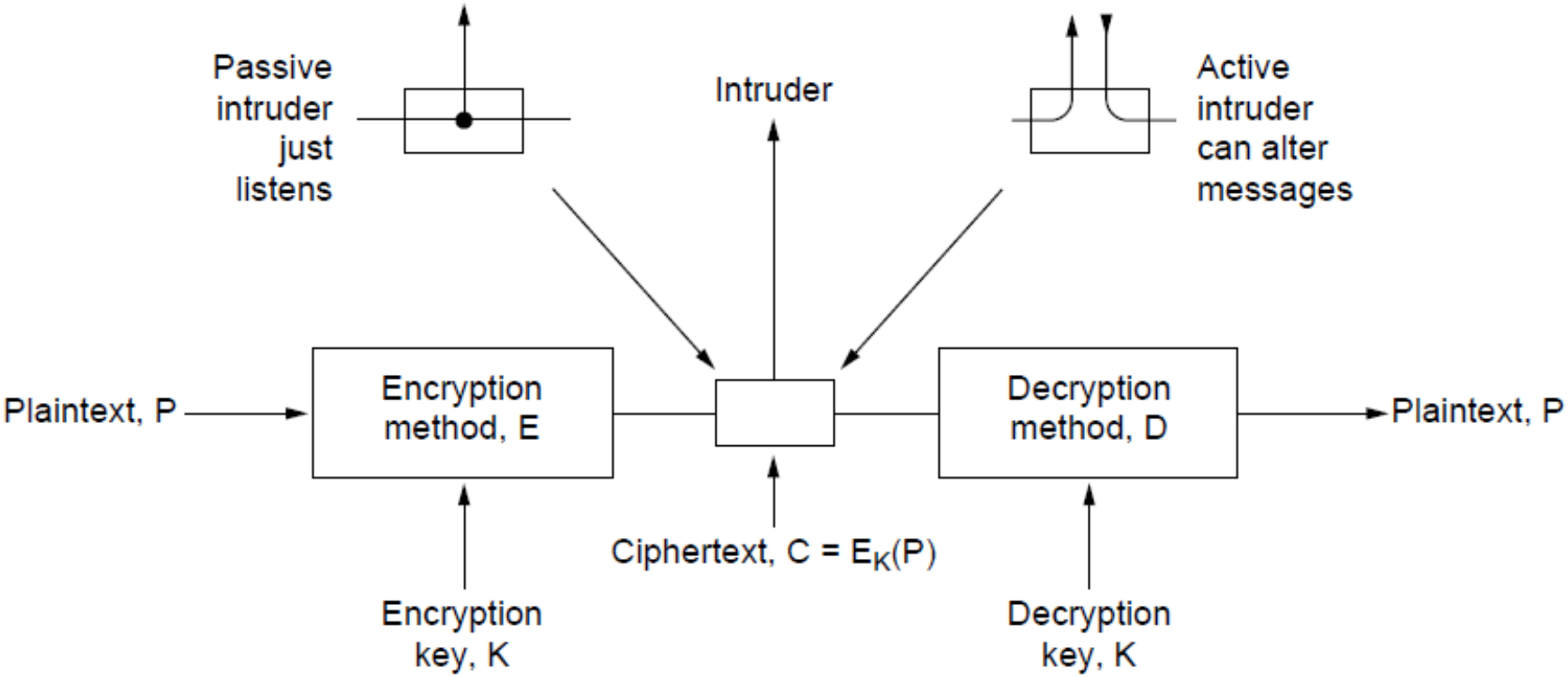
- **Cryptography** comes from the Greek words for "secret writing."
- A **cipher** is a character-for character or bit-for-bit transformation, without regard to the linguistic structure of the message.
- In contrast, a **code** replaces one word with another word or symbol. Codes are not used any more, although they have a glorious history.
- The messages to be encrypted, known as the **plaintext**, are transformed by a function that is parameterized by a **key**.
- The output of the encryption process, known as the **ciphertext**, is then transmitted, often by messenger or radio.
- We assume that the enemy, or **intruder**, hears and accurately copies down the complete ciphertext.

- Sometimes the intruder can not only listen to the communication channel (**passive intruder**) but can also record messages and play them back later, inject his own messages, or modify legitimate messages before they get to the receiver (**active intruder**).
- The art of breaking ciphers, called **cryptanalysis**, and the art devising them (cryptography) is collectively known as **cryptology.**
- Ciphertext $C = EK(P)$ to mean that the encryption of the plaintext P using key K gives the ciphertext C.
- Similarly, $P = DK(C)$ represents the decryption of C to get the plaintext again.

DK(Ek(P))=P

- The idea that the cryptanalyst knows the algorithms and that the secrecy lies exclusively in the keys is called **Kerckhoff's principle**, named after the Flemish military cryptographer Auguste Kerckhoff who first stated it in 1883 (Kerckhoff, 1883).
- ***Kerckhoff's principle:*** ***All algorithms must be public; only the keys are secret***

Introduction



The encryption model (for a symmetric-key cipher).

Encryption methods have historically been divided into two categories:

1. Substitution ciphers and
2. Transposition ciphers.

In a **substitution cipher** each letter or group of letters is replaced by another letter or group of letters to disguise it.

One of the oldest known ciphers is the **Caesar cipher**, attributed to Julius Caesar. In this method, *a* becomes *D*, *b* becomes *E*, *c* becomes *F*, ... , and *z* becomes *C*.

For example, *attack* becomes *DWWDFN*. In examples, plaintext will be given in lower case letters, and ciphertext in upper case letters.

- The general system of symbol-for-symbol substitution is called a **monoalphabetic substitution**, with the key being the 26-letter string corresponding to the full alphabet.
- For the key above, the plaintext *attack* would be transformed into the ciphertext *QZZQEA*.
- At first glance this might appear to be a safe system because although the cryptanalyst knows the general system (letter-for-letter substitution), he does not know which of the 26! ~ 4 x 1026 possible keys is in use. In contrast with the Caesar cipher, trying all of them is not a promising approach. Even at 1 nsec per solution, a computer would take 1010 years to try all the keys.

plaintext:	a b c d e f g h i j k l m n o p q r s t u v w x y z
ciphertext:	Q W E R T Y U I O P A S D F G H J K L Z X C V B N M

Monoalphabetic substitution

- **Substitution ciphers** preserve the order of the plaintext symbols but mask them.
- **Transposition ciphers,** in contrast, reorder the letters but do not mask them.
- The below figure depicts a common transposition cipher, the columnar transposition.
- The cipher is keyed by a word or phrase not containing any repeated letters. In this example, MEGABUCK is the key.
- The **plaintext is written horizontally**, in rows, padded to fill the matrix if need be.
- The **ciphertext is read out by columns**, starting with the column whose key letter is the lowest.
- The number of columns, k, is small, each of the k(k - 1) column pairs can be examined to see if its diagram frequencies match those for English plaintext.

Transposition Ciphers

M	E	G	A	B	U	C	K
7	4	5	1	2	8	3	6
p	l	e	a	s	e	t	r
a	n	s	f	e	r	o	n
e	m	i	l	l	i	o	n
d	o	l	l	a	r	s	t
o	m	y	s	w	i	s	s
b	a	n	k	a	c	c	o
u	n	t	s	i	x	t	w
o	t	w	o	a	b	c	d

Plaintext

pleasetransferonemilliondollarsto
myswissbankaccountsixtwotwo

Ciphertext

AFLLSKSOSELAWAIATOOSSCTCLNMOMANT
ESILYNTWRNNTSOWDPAEDOBUOERIRICXB

Some transposition ciphers accept a fixed-length block of input and produce a fixed-length block of output.

One-Time Pads (1)

Message 1:	1001001	0100000	1101100	1101111	1110110	1100101	0100000	1111001	1101111	1110101	0101110
Pad 1:	1010010	1001011	1110010	1010101	1010010	1100011	0001011	0101010	1010111	1100110	0101011
Ciphertext:	0011011	1101011	0011110	0111010	0100100	0000110	0101011	1010011	0111000	0010011	0000101
Pad 2:	1011110	0000111	1101000	1010011	1010111	0100110	1000111	0111010	1001110	1110110	1110110
Plaintext 2:	1000101	1101100	1110110	1101001	1110011	0100000	1101100	1101001	1110110	1100101	1110011

The use of a one-time pad for encryption and the possibility of getting any possible plaintext from the ciphertext by the use of some other pad.

One-Time Pads (2)

Bit number	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	
Data	1	0	0	1	1	1	0	0	1	0	1	0	0	1	1	0	
(a)	↘	↕	↕	↘	↔	↘	⤢	⤢	↔	↕	↔	⤢	⤢	↔	↔	↕	What Alice sends
(b)	+	+	×	×	×	+	+	×	+	×	+	×	×	×	+	×	Bob's bases
(c)	↕	↕	⤢	↘	↘	↕	↔	⤢	↔	⤢	↔	⤢	⤢	↘	↔	⤢	What Bob gets
(d)	No	Yes	No	Yes	No	No	No	Yes	Yes	No	Yes	Yes	Yes	No	Yes	No	Correct basis?
(e)		0		1				0	1		1	0	0		1		One-time pad
(f)	×	+	+	×	+	+	×	+	+	×	×	+	×	+	×	×	Trudy's bases
(g)	x	0	x	1	x	x	x	?	1	x	?	?	0	x	?	x	Trudy's pad

An example of quantum cryptography